

자료구조및실습: 2018년 1학기

제 13 주:

리스트 반복자



강 지 훈

jhkang@cnu.ac.kr

충남대학교 컴퓨터공학과



© J.-H. Kang, CNU

[제 13 주]

리스트 반복자

실습 목표

이론적 관점

- 구현에 독립적인 설계 - 캡슐화
- 반복자의 개념
- 자료형으로서의 Interface

구현적 관점

- 반복자의 사용법과 구현 방법
- Interface 를 자료형으로 사용하는 방법

반복자 (Iterator)

리스트의 사용과 구현

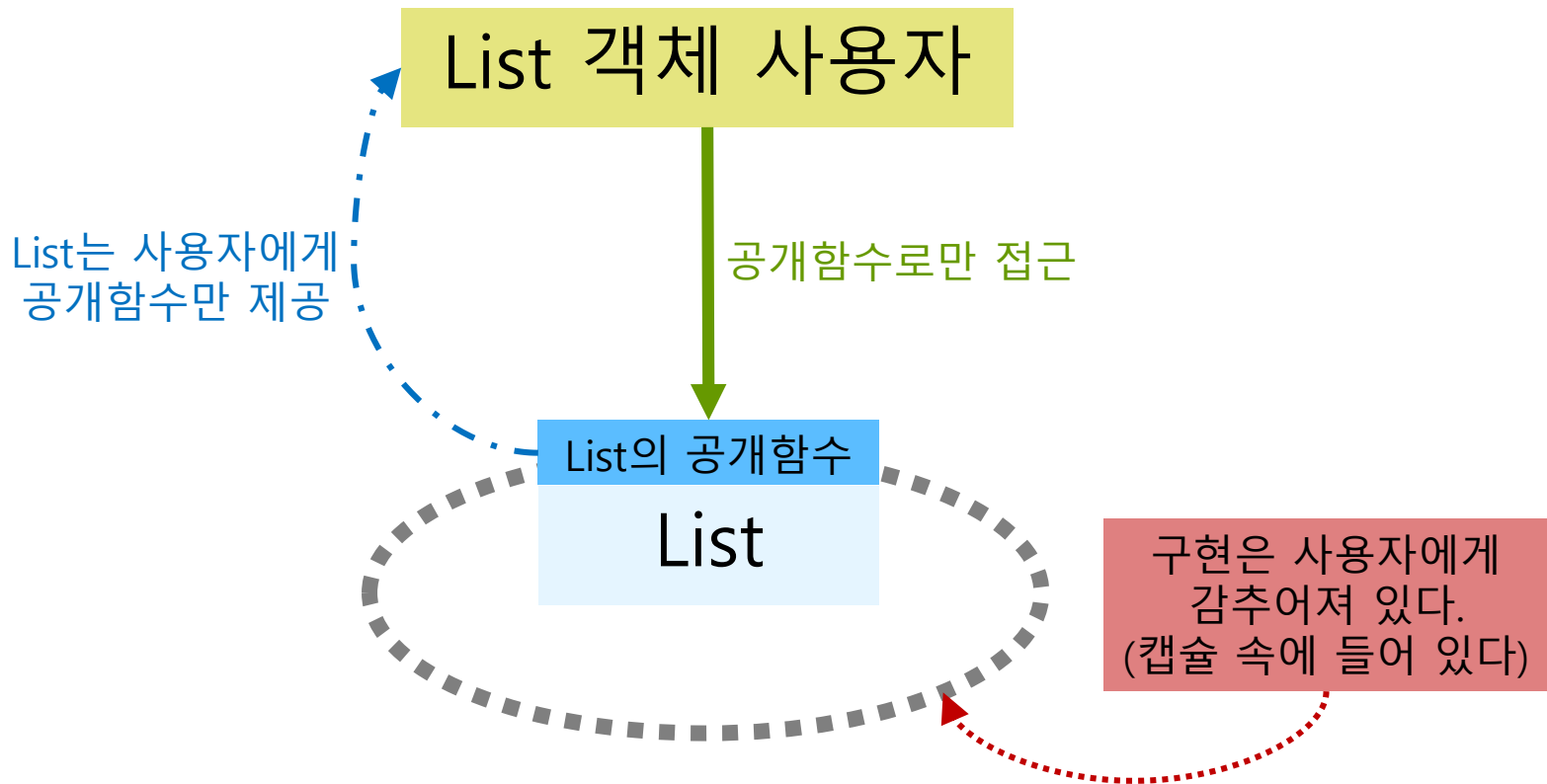


□ 리스트를 사용해 보자

- 원소가 정수형인 리스트를 이용한다.
- 키보드의 입력에 따라 다양한 일을 실행한다.
 - 리스트의 생성, 소멸, 삽입, 삭제, 검색 등의 일을 실행한다.
- 사용자의 관점에서 구현에 독립적이라는 것은?
 - 구현의 독립성은 왜 필요한가?

구현의 독립성

캡슐화 (Encapsulation)



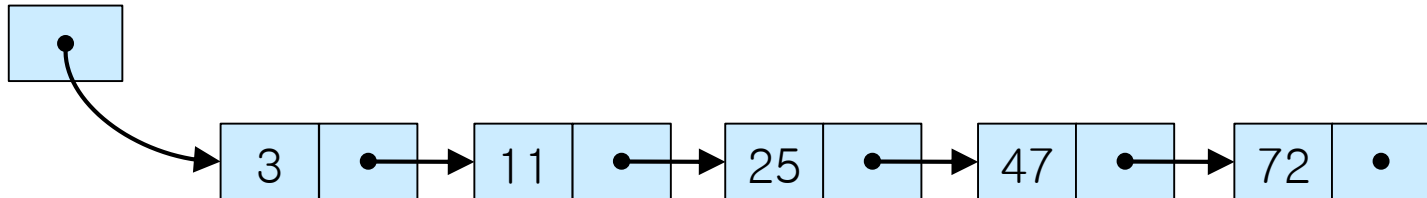
□ 동일한 일을 하는 서로 다른 구현

- List를 두 가지 다른 방법으로 각각 구현한다.

- Sorted Array



- Sorted Linked List

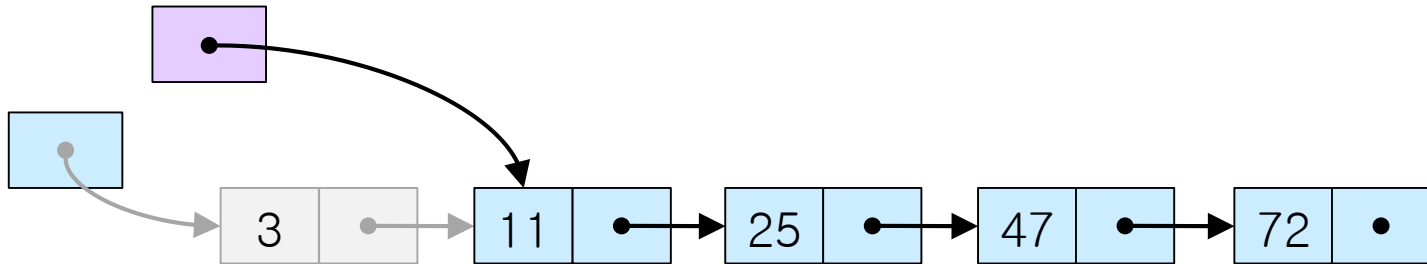
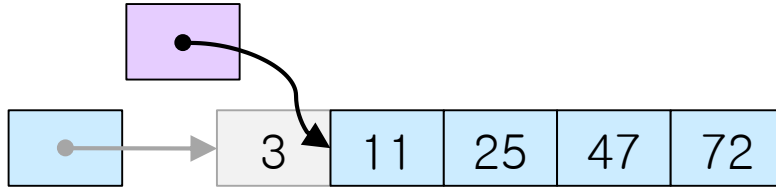


- 행위 별로 각각의 구현의 장단점을 이해한다.

- 삽입은?
- 삭제는?
- 다른 행위들은 ?

□ Recursive Structure 를 갖고 있는 List

- 하나를 떼어내고 생각해도 여전히 리스트.



■ Recursive algorithm으로!!

- 리스트 전체를 대상으로 실행되는 일들이라면.
 - ◆ 예: 최대값, 합계, 출력, 역순으로 출력,
- 그 구현이 배열이든, 연결리스트이든.

반복자의 개념, 사용과 구현

□ 사용자가 리스트를 스캔 할 필요가 있다면?

■ 구현과 무관하게:

- 리스트의 구현 방법을 몰라도, 아래의 표현이 가능한가?
 - ◆ 배열을 사용했는지, 아니면 연결리스트를 사용했는지?

```
int pass = 0 ;
int x = 0 ;
while ( x < list._element.size() ) {
    if ( list._element[x].score() >= 60 )
        pass++ ;
    x++ ;
}
```

```
int pass = 0 ;
Node x = list._head ;
while ( x != null ) {
    if ( x.element().score() >= 60 )
        pass++ ;
    x = x.next() ;
}
```

- 사용자는 차례대로 리스트의 원소의 값을 알고 싶을 뿐!

과제에서 해결할 문제



[과제 13-1]

리스트를 배열로 구현

□ [프로그램 13]

- 동일한 두 개의 프로그램을 작성한다.
 - List 의 구현은 서로 다르게 한다.
 - ◆ Sorted Array List
 - ◆ Sorted Linked List
 - 응용을 위한 App 은 동일하다.
 - ◆ 선언을 위한 부분 몇 군데만 수정한다.

□ 입출력

■ 입력:

- 이용하여 키보드에서 문자를 반복하여 입력 받는다.
- '!' 가 입력되면 프로그램을 종료한다.

■ 출력:

- 입력된 문자에 따라 미리 정의된 출력을 한다.

문제 [1]

입력 문자에 따라 해야 할 일

- '%': 정수입력을 받는다. 받은 값은 리스트에 삽입한다.
 - ◆ 이미 존재하는지 미리 확인하여, 존재하지 않으면 삽입한다.
 - 존재하면, 삽입하지 않고 이미 존재한다는 오류 메시지를 내보낸다.
 - ◆ 삽입 전에 미리 검사하여, 리스트가 Full 이면 삽입할 수 없다는 메시지를 내보낸다.
 - ◆ 삽입 후에 정상적으로 삽입이 되었는지를 메시지로 출력한다.
- '~': 리스트를 reset, 즉 empty로 만든다.
- '-': 리스트의 최소값을 삭제하고, 그 값을 출력한다.
 - ◆ 삭제 전에 미리 검사하여, Empty 이면 삭제할 수 없다는 메시지를 내보낸다.
- '+': 리스트에서 최대값을 삭제하고, 알려준다.
 - ◆ 삭제 전에 검사하여, empty 이면 오류 메시지를 내보낸다.

□ 문제 [2]

■ 입력 문자에 따라 해야 할 일 (계속)

- '#': 리스트의 길이를 출력한다.
- '?': 정수를 입력 받는다. 숫자에 해당하는 크기 순서에 해당하는 리스트 상의 값을 찾아 삭제하고, 그 원소를 출력한다.
 - ◆ 숫자가 0보다 작거나 또는 리스트의 개수보다 크거나 같으면 찾을 수가 없으므로, 적절한 오류 메시지를 내보내고, 숫자를 다시 입력 받는다.
- '/': 리스트의 원소를 처음부터 차례대로 출력한다.
- '!': 프로그램을 종료한다.

□ 이 과제에서 필요한 객체는?

- ApplicationController

- AppView

- Model

- Interface List<E>
- Class SortedArrayList<E> implements List<E>
- Class SortedLinkedList<E> implements List<E>
- Class Node<E>

□ 출력의 예

<리스트를 시작합니다>

> 문자를 입력하시오 : %

-숫자를 입력하시오 : 24

[Insert] 삽입된 원소는 24입니다.

> 문자를 입력하시오 : %

-숫자를 입력하시오 : 78

[Insert] 삽입된 원소는 78입니다.

> 문자를 입력하시오 : %

-숫자를 입력하시오 : 132

[Insert] 삽입된 원소는 132입니다.

> 문자를 입력하시오 : %

-숫자를 입력하시오 : 99

[Insert] 삽입된 원소는 99입니다.

> 문자를 입력하시오 : %

-숫자를 입력하시오 : 51

[Insert] 삽입된 원소는 51입니다.

> 문자를 입력하시오 : %

-숫자를 입력하시오 : 5

[Insert] 삽입된 원소는 5입니다.

> 문자를 입력하시오 : %

-숫자를 입력하시오 : 2

[Insert] 삽입된 원소는 2입니다.

> 문자를 입력하시오 : /

[LIST] 2 5 24 51 78 99 132

> 문자를 입력하시오 : +

[Delete] 삭제된 원소는 132입니다.

> 문자를 입력하시오 : ?

-숫자를 입력하시오 : 2

[Delete] 삭제된 원소는 24입니다.

> 문자를 입력하시오 : #

[Length] 리스트에는 현재 5개가 있습니다.

> 문자를 입력하시오 : -

[Delete] 삭제된 원소는 2입니다.

> 문자를 입력하시오 : ~

- 리스트를 비웠습니다.

> 문자를 입력하시오 : /

[LIST]

> 문자를 입력하시오 : !

<리스트가 끝났습니다>

main()

□ main()을 위한 class

```
public class DS1_13_학번_이름 {  
    public static void main(String[] args) {  
        AppController appController = new AppController() ;  
        appController.run() ;  
    }  
}
```

Class "AppController"

□ ApplicationController: 인스턴스 변수, 생성자

```
public class ApplicationController {
    // 비공개 변수들
    private AppView _appView ;
    private List<Integer> _sortedList ;

    // 생성자
    public ApplicationController()
    {
        this._appView = new AppView() ;
    }

    // 비공개함수의 구현
    .....

    // 공개함수의 구현
    .....
} // End of class "AppController"
```

첫 번째 과제는, 배열로 구현한 리스트를 사용한다. 그러므로 리스트의 class로 "SortedList" 를 사용한다. 두 번째 과제는, 첫 번째 과제와 동일한 문제이면서 단지 연결체인으로 구현한 리스트를 사용한다.

그러므로 자료형은 interface "List<>", 인스턴스 변수 이름은 "_sortedList"로, 동일하게 사용한다. 유일하게 달라지는 부분은 실제 객체 생성하는 곳만 달라진다. (함수 run()의 첫번째 줄의 리스트 객체 생성하는 것을 볼 것)

□ ApplicationController: 비공개 함수의 구현 [1]

- private void showSize()
 - `_sortedList`의 `size()` 를 사용
 - `_appView`의 `outputSize()` 메소드를 사용
- private void reset()
 - `_sortedList`의 `clear`메소드를 사용
- private void showAll()
 - `_sortedList`의 `elementAt()` 을 이용하여 구현 한다.
 - 이후 `Iterator`를 구현 한 뒤 수정
- private void add(int inputValue)
 - 삽입할 `Integer` 변수를 생성 (왜 `Integer`여야 할까?)
 - `_sortedList` 의 `add()` 를 실행
 - `_appview` 의 `outputAdd()` 메소드를 사용하여 출력

□ ApplicationController: 비공개 함수의 구현 [2]

■ 삭제 관련 함수들:

- `private void removeMin() {...}`
- `private void removeMax() {...}`
- `private void removeFrom (int aPosition) {
 this._appView.outputRemove (
 this._sortedList.removeFrom (aPosition));
}`
- 세 function 모두 `_sortedList` 의 적절한 메소드를 사용하여 구현한다.

□ ApplicationController: 공개 함수 run() 의 구현

```

public void run()
{
    this._sortedList = new SortedArrayList<Integer>();
    char command = 0;
    int input;

    this.showMessage(MessageID.Notice_StartProgram);
    while(command != '!') {
        command = this._appView.inputCharacter();
        if(command == '%'){
            input = this._appView.inputNumber();
            this.add(input);
        }
        else if(command == '~')
            this.reset();
        else if(command == '-')
            this.removeMin();
        else if(command == '+')
            this.removeMax();
        else if(command == '#')
            this.showSize();
        else if(command == '?') {
            input = this._appView.inputNumber();
            this.removeFrom(input);
        }
        else if(command == '/')
            this.showAll();
        else if(command == '!')
            break;
        else
            this.showMessage(MessageID.Error_WrongMenu);
    }
    this.showMessage(MessageID.Notice_EndProgram);
}

```

Class "AppView"

□ Class AppView

```
public class AppView {  
    // 비공개 상수/변수들  
    private Scanner _scanner ;  
  
    // 생성자  
    public AppView ()  
    {  
        this._scanner = new Scanner(System.in) ;  
    }  
  
    // 공개함수의 구현  
    .....  
}
```

□ Class "AppView" 의 공개함수

- 입력을 위한 공개 함수
 - public String inputString()
 - public int inputInt()
 - public char inputCharacter()
 - public int inputNumber()

□ Class "AppView" 의 공개함수

■ 출력을 위한 공개 함수

- public void outputSize (int size)
- public void outputAdd (int anElement)
- public void outputRemove (int anElement)
- public void outputElement (int anElement)

- public void outputMessage (String aMessage)

Enum "MessageID"

□ Enum "MessageID"

```
public enum MessageID {  
    // Message IDs for Notices:  
    Notice_StartProgram, Notice_EndProgram,  
    Notice_Reset,  
    Notice_ShowStartList,  
    Notice_ShowEndList,  
  
    // Message IDs for Errors:  
    Error_WrongMenu,  
    Error_InputFull,  
    Error_Empty,  
  
} //End of enum "MessageID"
```



Interface "List"

□ List 의 공개 함수는? [1]

■ 입력 문자 별 해야 할 일

- '%': 정수입력을 받는다. 받은 값은 리스트에 삽입한다.
 - ◆ 원소 삽입 : `public boolean add(T anElement)`
 - ◆ 존재 확인: `public boolean contains(T anElement)`
 - ◆ Full 검사: `public boolean isFull()`
- '~': 리스트를 reset, 즉 empty로 만든다.
 - ◆ Reset : `public void clear()`
- '-': 리스트에서 최소값을 갖는 원소를 삭제하고, 그 값을 알려준다.
 - ◆ 최소값 삭제 : `public T removeMin()`
 - ◆ Empty 검사 : `public boolean isEmpty()`
- '+': 리스트에서 최대값을 갖는 원소를 삭제하고, 알려준다.
 - ◆ 최대값 삭제: `public T removeMax()`
 - ◆ Empty 검사: `public boolean isEmpty()`

□ List 의 공개 함수는? [2]

■ 입력 문자 별 해야 할 일 (계속)

- '#': 리스트의 길이를 출력한다.
 - ◆ 길이 얻기: `public int size()`
- '?': 정수를 입력 받는다. 숫자에 해당하는 크기 순서에 해당하는 리스트 상의 값을 찾아 삭제하고, 그 원소를 알려준다.
 - ◆ aPosition 위치의 원소 삭제: `public T removeFrom(int aPosition)`
 - ◆ 숫자가 0보다 작거나 또는 리스트의 개수보다 크거나 같으면 찾을 수가 없으므로, 적절한 오류 메시지를 내보내고, 숫자를 다시 입력 받는다.
- '/': 리스트의 원소를 처음부터 차례대로 알려준다.
 - ◆ 따로 생각하자!!

□ Interface "List" 의 공개함수[1]

- public void clear()
 - SortedList 를 초기화
- public int size()
 - SortedList 의 원소의 개수를 얻는다
- public boolean isFull()
 - SortedList 가 가득 차 있으면 true 를, 아니면 false 를 얻는다
- public boolean isEmpty()
 - SortedList 가 비어 있으면 true 를, 아니면 false 를 얻는다
- public boolean contains(T anElement)
 - anElement 의 값과 같은 값을 갖는 원소가 SortedList 에 존재하면 true 를, 아니면 false 를 얻는다

□ Interface "List" 의 공개함수[2]

- `public boolean add(T anElement)`
 - SortedList 에 `anElement` 를 삽입
 - 삽입이 성공할 경우 `true` 를, 실패하면 `false` 를 얻는다
- `public T removeMin()`
 - SortedList 에서 가장 작은 값을 갖는 원소를 삭제
 - 리스트가 비어 있으면 `null` 을, 아니면 삭제 된 최소값 원소를 얻는다
- `public T removeMax()`
 - SortedList 에서 가장 큰 값을 갖는 원소를 삭제
 - 리스트가 비어 있으면 `null` 을, 아니면 삭제 된 최대값 원소를 얻고,
- `public T removeFrom (int aPosition)`
 - SortedList 에서 `aPosition` 번째의 원소를 삭제
 - `aPosition` 의 값은 0 보다 크거나 같고 `size()` 보다 작아야 유효하다.
 - 리스트가 비어 있거나 `aPosition` 이 유효하지 않으면 `null` 을, 아니면 삭제 된 원소를 얻는다

interface List [1]

```
public interface List<T> {  
    public void clear();  
    public int size();  
  
    public boolean contains(T anElement);  
    public boolean isFull();  
    public boolean isEmpty();  
  
    public boolean add(T anElement);  
  
    public T removeMin();  
    public T removeMax();  
    public T removeFrom(int aPosition);  
}
```

Class "SortedList"의 구현



□ SortedArrayList: 인터페이스의 상속

```
public class SortedArrayList<T extends Comparable> implements List<T> {  
    private static final int DEFAULT_MAX_SIZE = 20 ;  
    private int _maxSize;  
    private int _size;  
    private T[] _element;
```

이 class "SortedArrayList" 는 interface "List"를 구현해야 함을 명시하고 있다.

즉, class "SortedArrayList" 에서는 interface "List"에 선언되어 있는 모든 함수를 구현해야 한다.

❑ SortedArrayList: Comparable 을 상속받은 generic type [1]

```
public class SortedArrayList<T extends Comparable> implements List {
    private static final int DEFAULT_MAX_SIZE = 20 ;
    private int _maxSize;
    private int _size;
    private T[] _element;
```

"extends" ?
"Comparable" ?

Interface "Comparable"

- Java에서 동일한 class 의 두 객체를 비교할 필요가 있을 때 사용한다.
- 우리의 경우, 리스트의 두 원소를 비교해야 한다.
- 단순한 숫자나 문자와는 달리, 객체를 비교하는 것은 객체에 따라 비교 방법이 달라질 수 있다.
- 그러므로 비교하는 객체들 즉 비교 대상 객체들의 class에 따라, 그에 합당하게 비교 방법을 제공해야 할 것이다.

❑ SortedArrayList: Comparable 을 상속받은 generic type [2]

```
public class SortedArrayList<T extends Comparable> implements List {
    private static final int DEFAULT_MAX_SIZE = 20 ;
    private int _maxSize;
    private int _size;
    private T[] _element;
```

"extends" ?
"Comparable" ?

Interface "Comparable" (설명 계속)

- 이를 위해 Java 는 "Comparable" 이라는 interface 를 제공하고 있다. 이 interface 는 "compareTo()" 라는 함수를 가지고 있다.
- 그러므로, 비교가 필요한 객체들의 class 는 interface "Comparable"을 구현해야 한다. 즉, "compareTo()" 함수를 구현해야 한다.

❑ SortedArrayList: Comparable 을 상속받은 generic type [3]

```
public class SortedArrayList<T extends Comparable> implements List {
    private static final int DEFAULT_MAX_SIZE = 20 ;
    private int _maxSize;
    private int _size;
    private T[] _element;
```

"extends" ?

"Comparable" ?

Interface "Comparable" (설명 계속)

- 이곳의 "T extends Comparable" 은, class "SortedArrayList" 에 주어진 generic type T 는 반드시 interface "Comparable"를 구현하고 있는 class 이어야 한다는 것을 의미한다. 즉, "SortedArrayList"를 사용할 때 generic type T 를 대체하여 주어지는 실제 class 는 반드시 interface "Comparable"을 구현하고 있어야 함을 강요하고 있다. (만일 T 가 실제 class로 대체될 때, "Comparable"을 구현하고 있지 않은 class 가 사용되고 있으면, Java 는 compile 오류를 발생시킨다.)
- 이렇게 함으로써, 이 class 안에서는 "compareTo()" 를 사용하여 type이 T 인 원소들 끼리 비교할 수 있다.

□ SortedArrayList: 생성자

■ public SortedArrayList()

- `_maxSize`를 `DEFAULT_MAX_SIZE` 로 초기화

■ public SortedArrayList(int givenMaxSize)

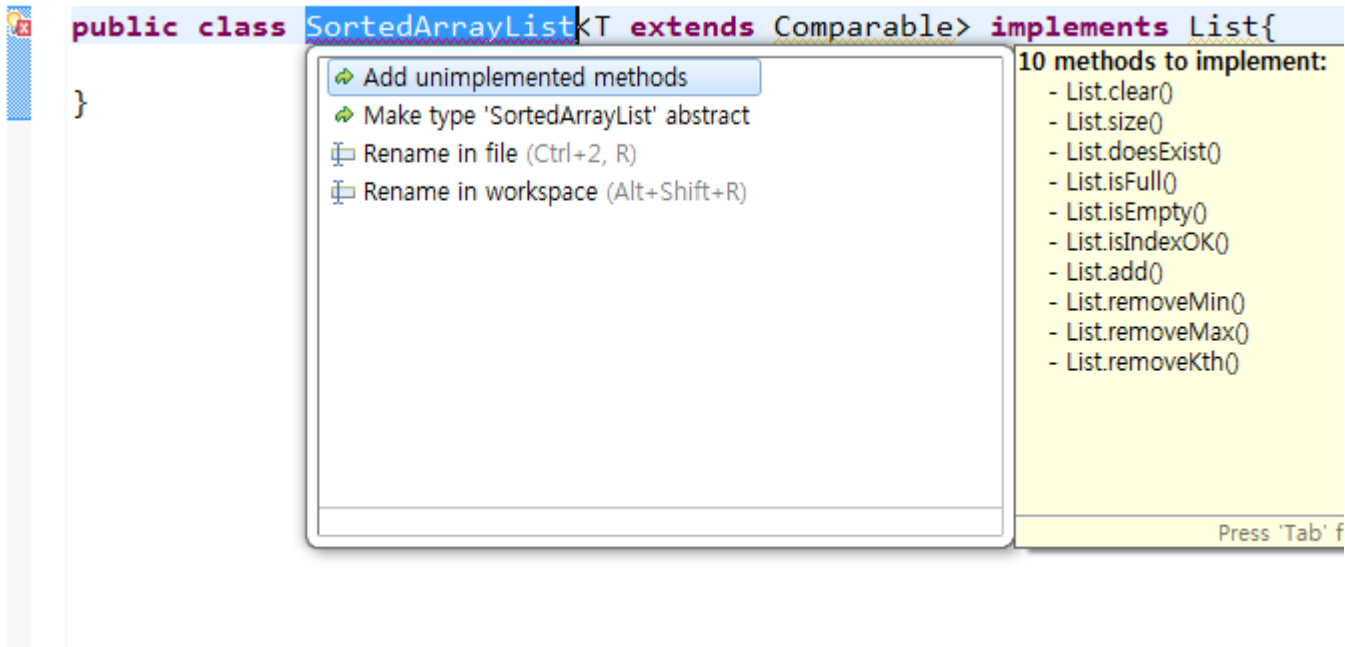
- `_maxSize`를 `givenMaxSize`로 초기화

■ 생성자 구현의 공통사항

- `_size` 를 0 으로 초기화
- `_elements[]` 배열 객체를 `_maxSize` 만큼 생성한다.
 - ◆ `Comparable` 은 interface 이다. Java 에서는 Interface 도 class 처럼 객체의 자료형으로 사용할 수 있다.
배열 원소의 자료형 T 가 `Comparable` 을 상속받고 있다. 그러므로 배열을 생성할 때 원소의 자료형이 `Comparable` 형인 배열로 생성하여야 한다.
 - ◆ `this._elements = new Comparable[this._maxSize];`

□ SortedArrayList: interface List 의 함수

- 사용할 모든 메소드는 interface List 에 선언되어 있다.
- Override 하여 구현한다.



☐ SortedArrayList:

- SortedArrayList의 Public Member function의 구현
 - public void clear()
 - ◆ `_size`를 0 으로 초기화
 - ◆ `_element[]` 의 모든 원소를 null 로 초기화
 - 배열의 크기는 당연히 `_maxSize` 이다.
 - public boolean isFull()
 - ◆ 배열 `_elements[]` 가 가득 차 있으면 true 를 돌려준다.
 - ◆ `_size` 가 `_maxSize` 와 같은 값을 갖고 있는지를 확인한다.
 - public boolean isEmpty()
 - ◆ 배열 `_elements[]` 가 비어 있으면 true 를 돌려준다.
 - ◆ `_size` 가 0 인지를 확인한다.
 - public int size()
 - ◆ `_size` 를 돌려준다.

❏ SortedArrayList: contains()

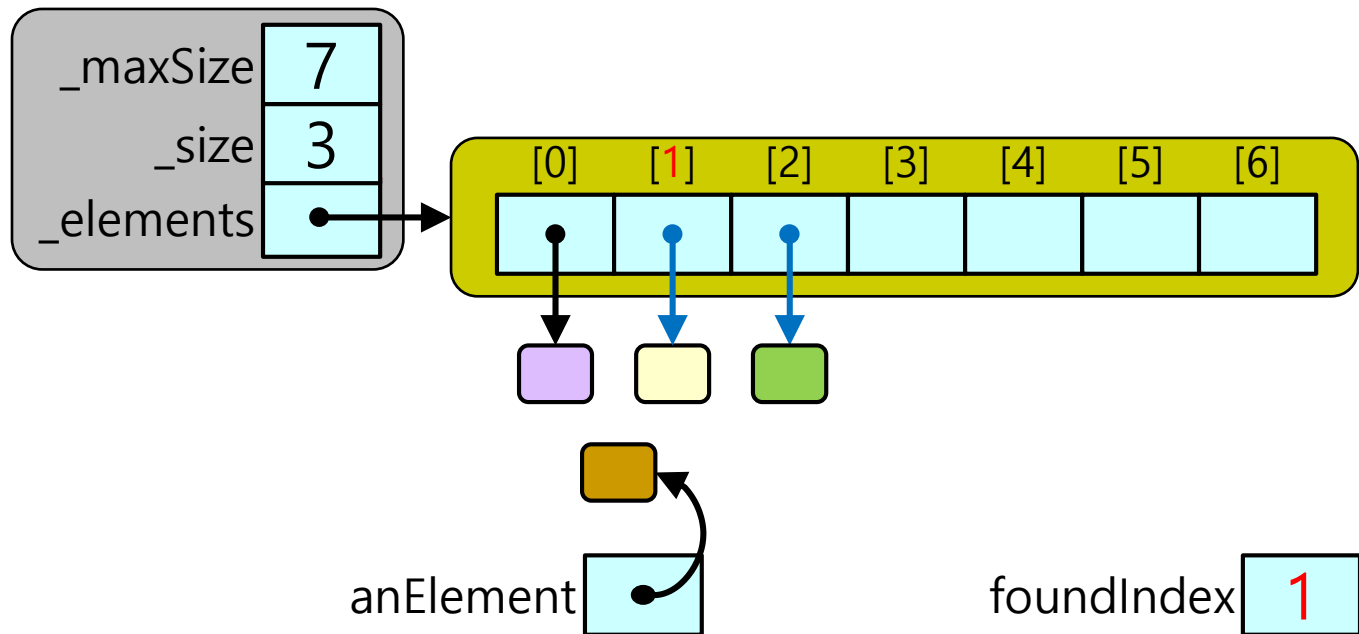
■ SortedArrayList의 Public Member function의 구현

```
public boolean contains(T anElement) {  
    // TODO Auto-generated method stub  
    boolean found = false;  
    for(int i = 0; i < this._size; i++) {  
        if(this._element[i].compareTo(anElement) == 0)  
            found = true;  
    }  
    return found;  
}
```

"compareTo()" 함수의 사용

□ SortedArrayList: add() [1]

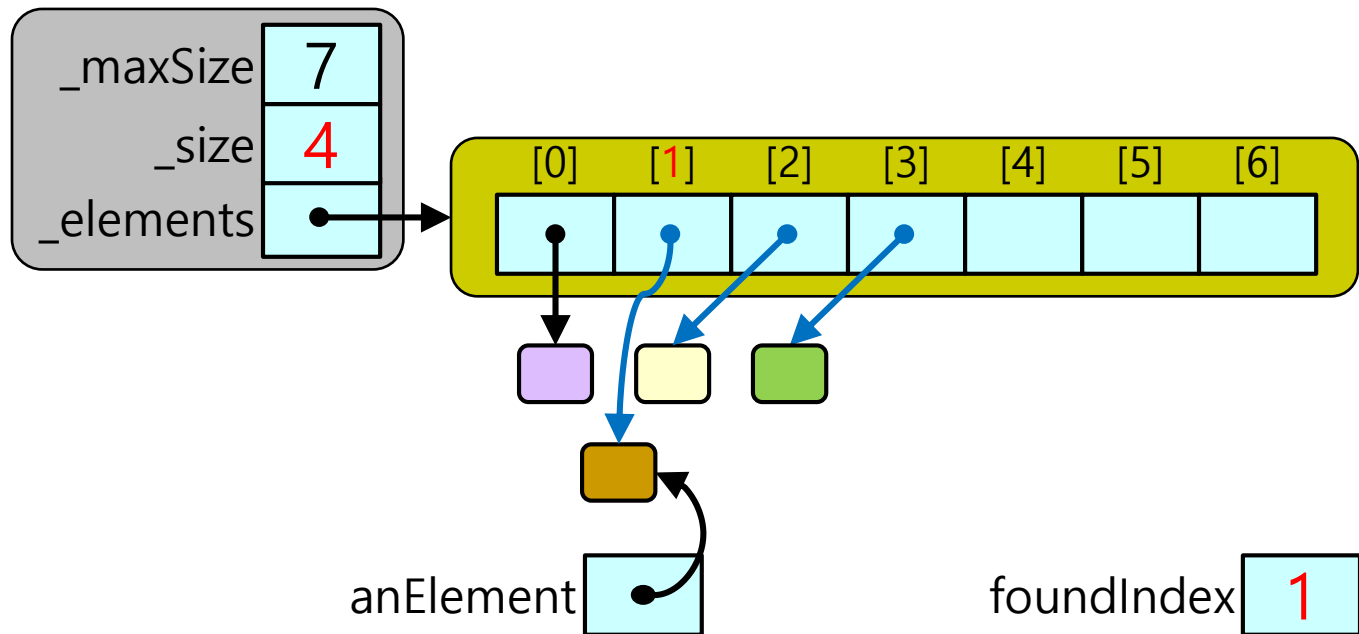
- public boolean add(T anElement)
 - compareTo() 를 이용하여 anElement 를 삽입할 위치를 찾는다.



□ SortedArrayList: add() [2]

■ public boolean add(T anElement)

- compareTo() 를 이용하여 anElement 를 삽입할 위치를 찾는다.
- 찾은 위치에 anElement 를 삽입한다.
- _size 를 증가시킨다.



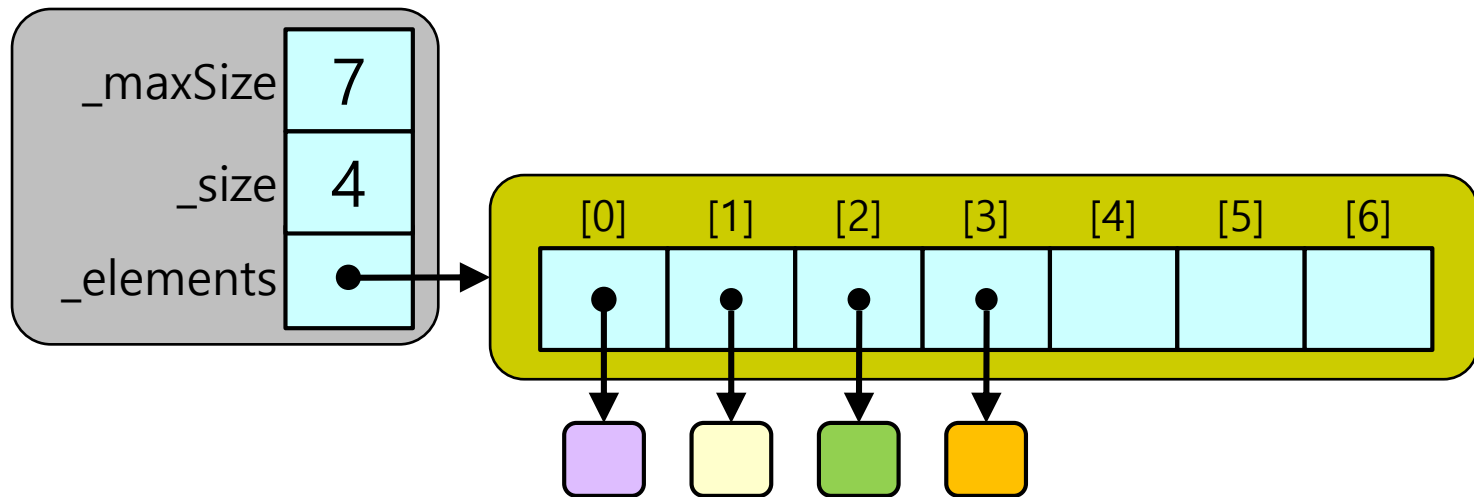
□ SortedArrayList: removeFrom()

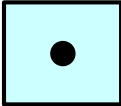
■ public T removeFrom(int aPosition)

- 리스트가 empty 인 경우:
 - ◆ null 을 돌려준다.
- 리스트가 empty 가 아닌 경우:
 - ◆ aPosition 이 this._size 보다 크거나 같거나 0 보다 작은 경우 null 을 돌려준다.
 - ◆ 삭제할 원소 (_elements[] 의 aPosition 번째 원소) 객체를 임시 저장 변수에 보관
 - ◆ aPosition 번째부터 (_size -1) 번째까지 원소들을 한 칸씩 앞으로 이동
 - ◆ _elements[] 의 (_size - 1) 번째의 원소 칸을 null 로 초기화
 - ◆ _size 를 하나 감소
 - ◆ 임시 저장 변수에 보관하였던 원소 객체를 돌려준다.

SortedArrayList: removeFrom() [1]

public T removeFrom(int aPosition)

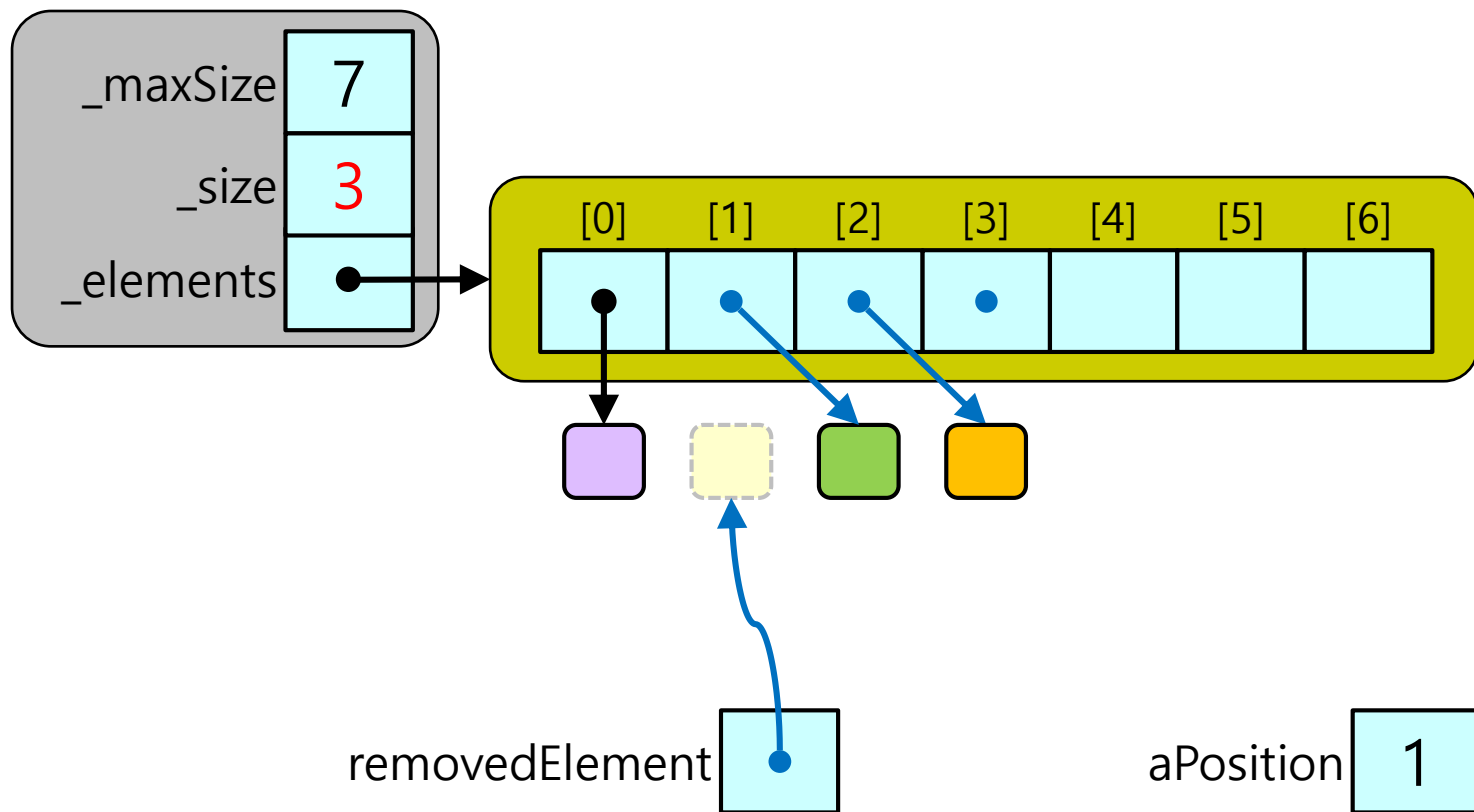


removedElement 

aPosition 

SortedArrayList: removeFrom() [2]

public T removeFrom(int aPosition)



❏ SortedArrayList: removeMin(), removeMax()

■ public T removeMin()

- min 값을 갖는 위치를 찾는다.
- 그 위치의 원소를 삭제하여 돌려준다.
- removeFrom() 을 참고하여 구현한다.

■ public T removeMax()

- removeMin() 을 참고하여 구현한다.

❏ SortedArrayList: elementAt()

- public T elementAt (int anOrder)
 - 주어진 순서 (anOrder)의 원소를 얻는다.
 - 리스트의 가장 앞 원소의 order 가 0 이다.
 - 리스트의 가장 뒤 원소의 order는 (_size-1) 이다.
- 구현에서의 유의할 점:
 - ◆ 주어진 anOrder 가 유효한 지를 검사한다.
 - 유효 범위: $0 \leq \text{anOrder} < _size$
 - ◆ 여기서 리스트의 empty 검사를 따로 해야 할까?

Interface "Iterator"



□ 반복의 추상화

- 구현에 독립적으로 list 원소들에 대해 순차 검색을 실행
- Iterator 를 class로 정의하고, 반복이 필요할 때마다 반복자 객체를 생성하여 반복에 사용

```

T                e ;
List<T>          list ;
Iterator<T>      iterator ;
// Iterator는 반복을 추상화 한,
// 반복을 위한 class
int pass = 0 ;
iterator = list.iterator() ;
while ( iterator.hasNext () ) {
    // 리스트의 원소를 얻어내어 사용
    e = iterator.next () ;
    if ( e.score() >= 60 ) {
        pass++ ;
    }
}

```

```

int pass = 0 ;
int x = 0 ;
while ( x < list._element.size() ) {
    if ( list._elements[x].score() >= 60 )
        pass++ ;
    x++ ;
}

```

```

int pass = 0 ;
Node<T> x = list._head ;
while ( x != null ) {
    if ( x.element().score() >= 60 )
        pass++ ;
    x = x.next() ;
}

```


□ Interface "Iterator" 의 공개 함수

■ Why interface?

- 원소를 모아 놓은 객체라면, 그 내부 구조가 어떤 형태라도 그 객체는 소유하고 있는 원소들에 대해 반복이 필요할 수 있다.
- 이러한 원소를 모아 놓는 객체의 class 라면, 어느 것이라도 반복의 기능을 추가할 수 있으면 좋을 것이다.

■ Interface Iterator<T>

- public boolean hasNext ()
 - ◆ 다음 값이 있는지 확인 하는 메소드
- public T next ()
 - ◆ 다음 값을 받아오는 메소드

□ Interface "Iterator"

```
public interface Iterator<T> {  
    public boolean hasNext () ;  
    public T next () ;  
}
```

Interface “Iterator” 의 구현

□ SortedArrayList 예서의 반복자 구현

■ Private Attributes

```
public class ListIterator<T> implements Iterator<T>
{
    private int _nextPosition ; //현재 (노드) 위치
```

■ Public Functions

```
private ListIterator()
{
    this._nextPosition = 0 ;
}
```

□ List가 배열로 구현된다면? [2]

■ Public Functions

```
public boolean hasNext() {
    return ( this._nextPosition <
             SortedArrayList.this.size() );
}
```

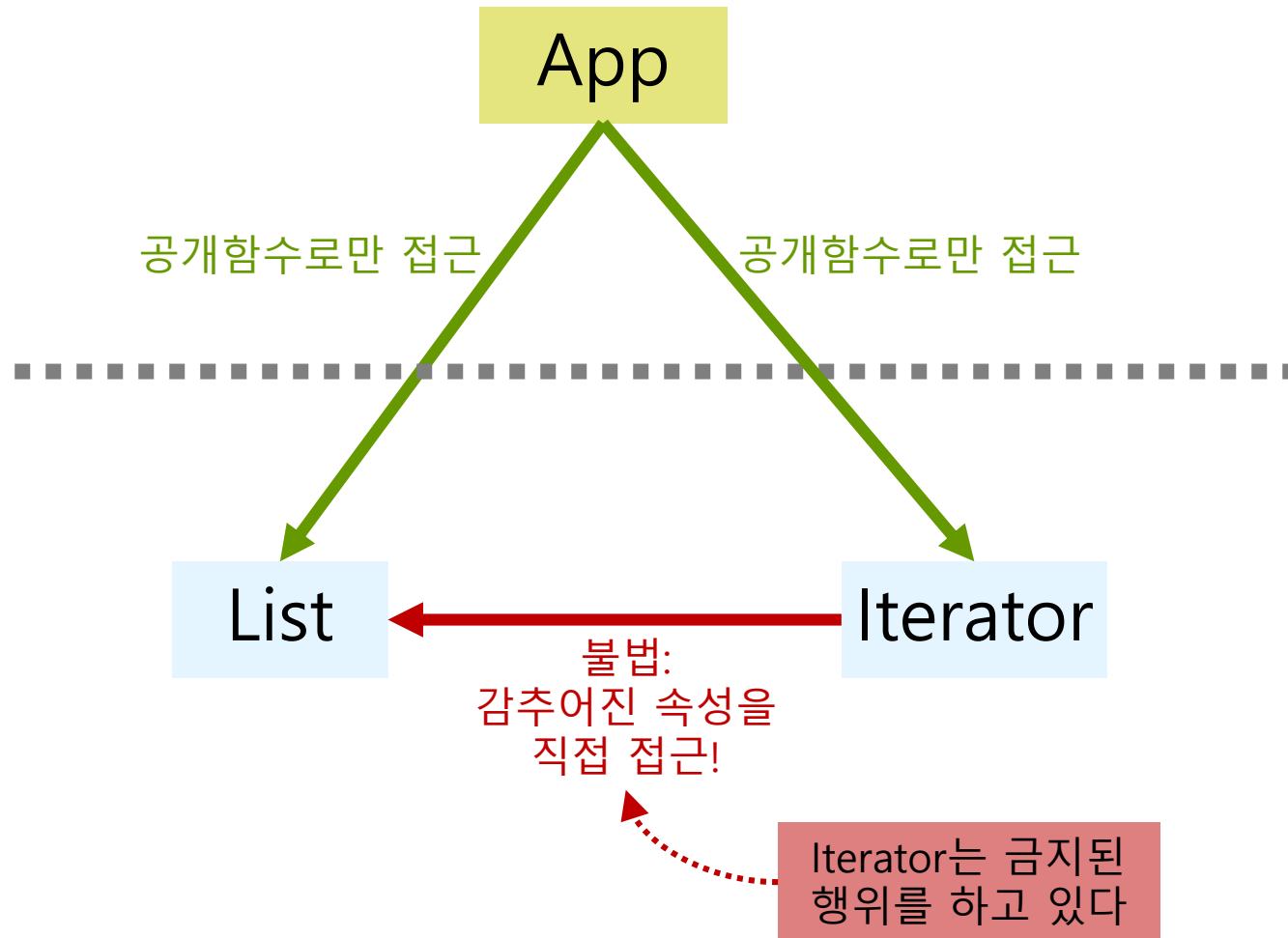
```
public T next() {
    if ( this._nextPosition ==
         SortedArrayList.this.size() )
    {
        return null ;
    }
    else {
        T element = (T) SortedArrayList.this._elements[this._nextPosition] ;
        this._nextPosition++ ;
        return element ;
    }
}
```

```
int pass = 0;
int index = 0;
while ( index < list._size ) {
    if (list._elements[index] >= 60)
        pass++;
    index++;
}
```

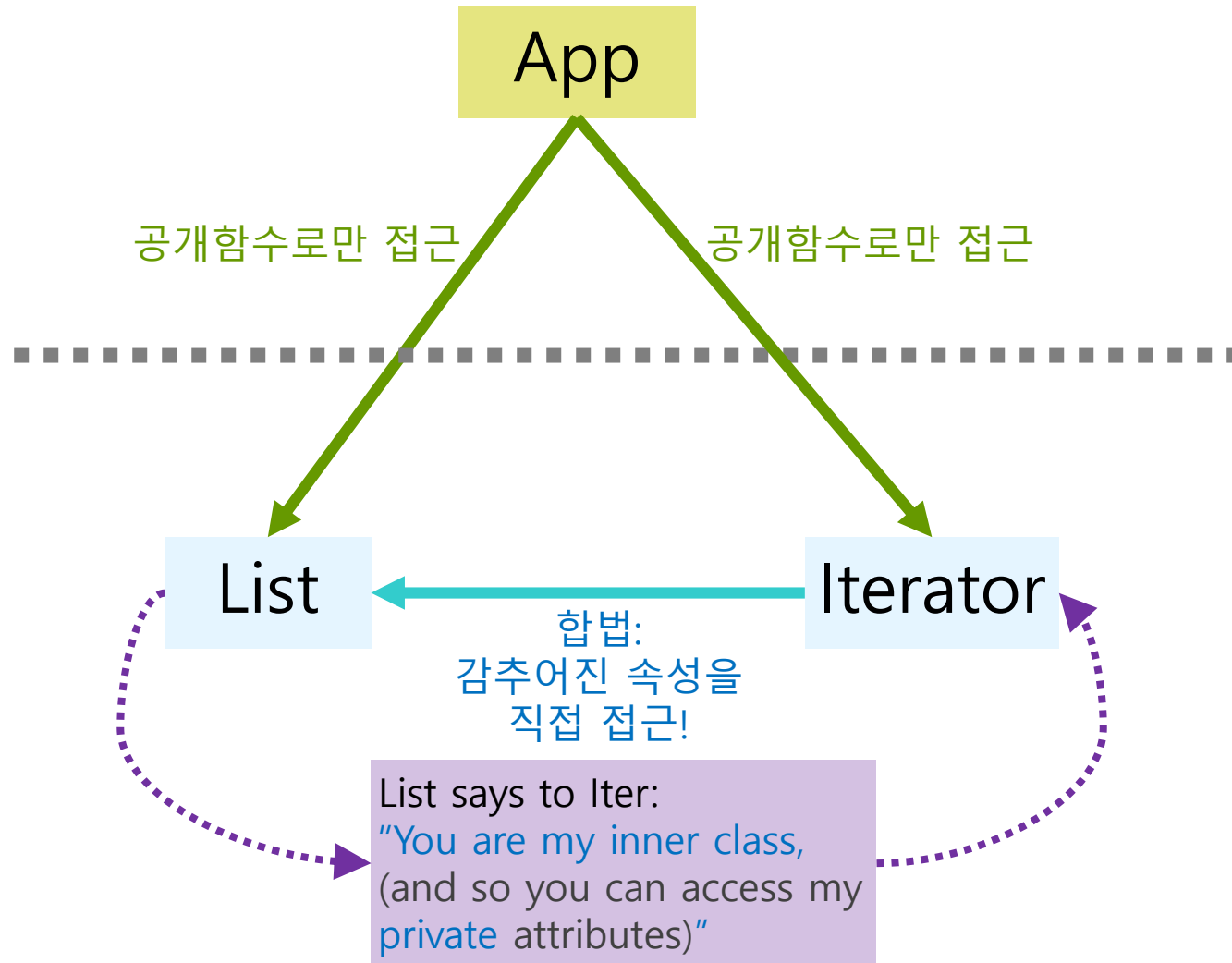


```
int pass = 0;
Iterator<T> iterator = list.iterator() ;
while ( iterator.hasNext() ){
    if ( iterator.next() >= 60)
        pass++;
}
```

□ List 와 Iterator 의 관계는?



□ List 와 Iterator 는 친구?



SortedArrayList:

```

@SuppressWarnings({ "unchecked", "rawtypes" })
public ListIterator<T> listIterator()
{
    return new ListIterator();
}
@SuppressWarnings({ "rawtypes", "hiding" })
public class ListIterator<T> implements Iterator
{
    private int _nextPosition;

    private ListIterator()
    {
        this._nextPosition = 0;
    }
    public boolean hasNext()
    {
        return (this._nextPosition < size());
    }
    public T next()
    {
        if(this._nextPosition == size()) {
            return null;
        }
        else {
            @SuppressWarnings("unchecked")
            T t = (T)_element[this._nextPosition];
            this._nextPosition++;
            return t;
        }
    }
}

```


□ Class ApplicationController 의 showAll() 의 수정

- List의 원소를 출력하기 위해 사용했던 elementAt() 대신에, Iterator 를 사용하여 구현한다.

```
public void showAll () {  
    this.showMessage (MessageID.Notice_ShowStartList) ;  
    Iterator<Integer> iterator = this._sortedList.listIterator() ;  
    while ( iterator.hasNext() ) {  
        this._appView.outputElement( (int) iterator.next()) ;  
    }  
    this.showMessage (MessageID.Notice_showEndList) ;  
}
```

자료형으로 interface "Iterator" 사용:

SortedLinkedList 를 사용 할 때도 수정하지 않아도 된다.

[과제 13-2]

리스트를 연결 체인으로 구현

Class "Node" 의 구현



□ Node: 인스턴스 변수

```
public class Node <T> {  
    private T        _element;  
    private Node<T>  _next;
```

□ Node: 생성자, Getter/Setter

- 생성자: 초기 값으로 주어지는 매개변수에 따라 3 가지 유형을 제공

- `public Node ()`
 - ◆ 아무 것도 주어지지 않은 기본적인 생성자
- `public Node (T givenElement)`
 - ◆ 원소만 주어지는 경우에 사용
- `public Node (T givenElement, Node givenNext)`
 - ◆ 원소와 다음 노드가 모두 주어지는 경우에 사용

■ Getter/Setter

- `public T element()`
 - ◆ `_element` 를 반환
- `public void setElement(T newElement)`
 - ◆ `newElement` 를 `_element` 에 저장
- `public Node<T> next()`
 - ◆ `_next` 를 반환
- `public void setNext(Node<T> newNext)`
 - ◆ `newNext` 를 `_next` 에 저장

Class "SortedLinkedList" 의 구현

□ SortedLinkedList: 인스턴스 변수

```
public class SortedLinkedList<T extends Comparable> implements List<T>
{
    private int _size ;
    private Node<T> _head ;
```

□ SortedLinkedList: 생성자, 공개함수

■ 생성자:

- public SortedLinkedList()
 - ◆ _size 를 초기화
 - ◆ _head 를 초기화

■ 공개함수

- public void clear()
 - ◆ _size 를 0 으로 초기화
 - ◆ _head 를 null 로 초기화
- public boolean isFull()
 - ◆ 언제나 false 를 돌려준다
- public boolean isEmpty()
 - ◆ _size 를 확인하여 0 이면 true 를 돌려준다

❑ SortedLinkedList: contains(), size()

■ public boolean contains(T anElement)

- 모든 Node 를 _head 부터 시작하여 모든 노드의 원소를 확인하여 anElement 와 같은 값이 존재하는지를 확인한다.
- 원소를 비교할 때 compareTo() 를 사용한다

■ public int size()

- _size 를 반환

□ SortedLinkedList: add()

■ public boolean add (T anElement)

- 노드의 원소를 비교하여 연결 체인에서의 삽입 위치를 찾는다.
- 주어진 원소 anElement를 보관하는 새로운 노드를 생성한다.
- 이 노드를 삽입 위치에 삽입한다.
- 유의할 점:
 - ◆ 현재 상태가 empty 라면?
 - ◆ 삽입 위치가 맨 앞 이라면?
- 이전의 실습 과제들을 참고하여 코드를 완성할 것.

❏ SortedLinkedList: 원소의 삭제

■ public T removeFrom (int aPosition)

- 연결체인이 empty 이면, null 을 돌려준다.
 - ◆ 이 조건은 다음의 조건에 포함되므로, 별도로 점검하지 않아도 됨.
- aPosition 이 0 보다 작거나 this._size 보다 크거나 같은 경우 null를 돌려준다.
 - ◆ 연결 체인이 empty 인 경우에도 유효하게 점검이 가능함.
- 유의할 점:
 - ◆ 삭제할 노드가 맨 앞 노드라면?

■ public T removeMin ()

- 리스트가 empty 이면 null 을 돌려준다.
- removeFrom() 의 경우를 참고하여 구현한다.

■ public T removeMax ()

- removeMin() 의 경우를 참고하여 구현한다.

■ 이전의 실습 과제들을 참고하여 코드를 완성할 것.

Interface “Iterator” 의 구현

□ Linked List를 위한 반복자: 인스턴스 변수, 생성자

```

public class SortedLinkedList<T extends Comparable<T>>
    implements List<T>
{
    .....
    // SortedLinkedList 의 Inner Class
    public class ListIterator<T> implements Iterator<T>
    {
        private Node<T> _next ;

        // 생성자
        private ListIterator() {
            this._next = SortedLinkedList.this._head ;
        }

        .....
    } // End of Inner class "ListIterator"
} // End of Class " SortedLinkedList"

```

□ Linked List에서의 반복자 구현

■ Public Functions

```
public boolean hasNext()
{
    return ( this._next != null ) ;
}

public T next()
{
    if ( this._next == null ) {
        return null ;
    }
    else {
        T element = (T) this._next.element() ;
        this._next = this._next.next() ;
        return element ;
    }
}
```

```
int pass = 0;
Node<T> node = list._head;
while ( node != NULL) {
    if (node.element().score() >= 60)
        pass++;
    node = node.next();
}
```



```
int pass = 0;
Iterator<T> iterator = list.iterator() ;
while ( iterator.hasNext() ){
    if ( iterator.next() >= 60)
        pass++;
}
```

요약

□ [문제 13] 요약

■ List의 이해

- 기본 개념
- 구현
 - ◆ SortedArrayList
 - ◆ SortedLinkedList
- 구현의 독립성

■ Iterator 의 이해

- 개념
 - ◆ Protected Attributes
- 사용법
- 구현

■ 두 구현의 장단점을 각 행위 별로 논해 보자

- 특별히 시간 복잡도의 관점에서

■ Iterator 를 사용해 보고 느낀 점을 논해 보자

보고서 작성과 제출



□ PDF 보고서 작성 방법

■ 겉장

- 제목: 자료구조 실습 보고서
- [제 13 주] 리스트 반복자
- 제출일
- 학번/이름

■ 내용

1. 프로그램 설명서
 1. 주요 알고리즘 /자료구조 /기타
 2. 함수 설명서
 3. 종합 설명서
2. 프로그램 장단점 분석
3. 실행 결과 분석
 1. 입력과 출력 (화면 capture 시킬 것)
 2. 결과 분석
4. 소스코드

□ 과제 제출

■ 파일 이름 작명 방법

● DS1_13_학번_이름.zip

● 폴더의 구성

◆ DS1_13_학번_이름

■ 프로그램

– 프로젝트 폴더 / 소스

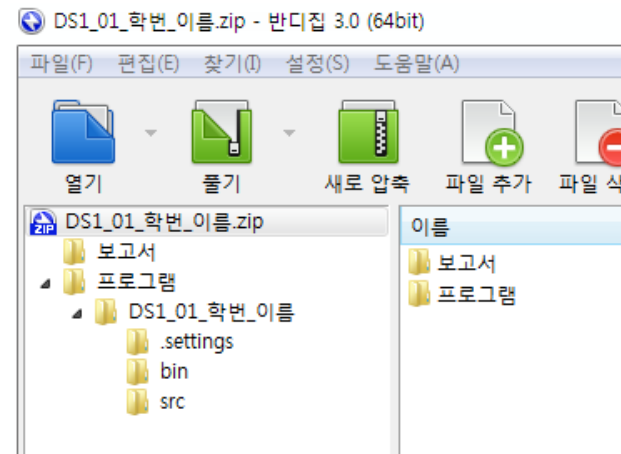
– 메인 클래스 이름 : DS1_13_학번_이름.java

■ 보고서

– 이곳에 보고서 문서 파일을 저장한다.

– 입력과 실행 결과는 화면 image로 문서에 포함시킨다.

– 문서는 pdf 파일로 만들어 제출한다.



[제 13 주] 실습 끝



